

Recognition Model — Training Results

Proactive Robotic Assistance for Interior Construction Assembly

ITECH M.Sc. Thesis · Recognition Layer · 21 September 2026

Summary

Five 15-fold leave-one-subject-out (LOSO) experiments were run on the ceiling-panel assembly dataset (93 takes, 15 subjects, 3 synchronised camera views). The evaluation protocol was rebuilt first, because the previously available split was found to be invalid.

Frame-level classification saturates at **macro-F1 $\approx$ 0.39** regardless of label encoding or augmentation. However, the metric that governs system behaviour is event-level trigger detection, and there the result is markedly better: **75% detection with a median latency of -0.3 s** — half of all firings occur ahead of the annotated onset, which is the behaviour proactive assistance requires.

The open problem is precision: **2.19 false alarms per true event**. The cause was traced to the training objective rather than to the trigger logic, and a corrective retrain is specified in Section 8.

1. Evaluation protocol corrections

Three defects were found and fixed before any model was trained. Each would have inflated or distorted reported performance.

1.1 The shipped data split leaked

All 15 subjects appeared in the train, validation and test directories simultaneously. Measured impact, same model and data, only the split changed:

random split	(same people in train and test)	balanced-acc 0.806
subject split	(disjoint people)	balanced-acc 0.298

All results in this report use subject-disjoint splits. Camera identity is deliberately excluded from the grouping key, since cam-05/06/07 record one performance from three angles and must never be separated.

1.2 Idle time was silently labelled as class 0

The label builder selects the per-frame class with a bare `idxmax`. On a frame where every class score is zero — genuine idle time — this returns the first column, Pull Cables. Measured across 1,064,615 frames:

```
frames with task_id_prob == 0 : 86,749 (8.15%) - all labelled class 0
share of Pull Cables frames that are actually idle : 61.9%
all other classes affected : 0.0%
idle runs are real gaps : median 4.1 s, max 6.6 s
```

`task_id_plateau_vector.max(axis=1) == 0` is an exact background detector (100.00% agreement over 286,123 frames) and is now used to mask these frames.

1.3 Tied label scores systematically erased one class

The plateau curve saturates at exactly 1.0, so when two tasks overlap both lanes hold 1.0. 18% of windows contain such ties, and argmax resolves every one toward the lowest class index. Place is index 6, so it lost every tie:

```
Place ground truth : 9.61% of windows -> 0.54% after argmax
after tie-break   : Align 2.90% -> 10.85%, Place 0.54% -> 3.44%
```

Ties are now broken with the peak-curve score, which has a unique maximum by construction. 11.2% of evaluation rows change.

2. The labels are multi-label

Lift is an umbrella annotation covering the whole carry-and-hang sequence, with Place (the waist-to-ceiling push) and Align (fore/aft positioning at the structure) nested inside it. Co-occurrence confirms this:

```
P(Lift active | Place active) = 94%
P(Lift active | Align active) = 86%
P(anything   | Screw active) = 0%
```

```
frames with 2 or more classes active simultaneously : 19.4%
```

Single-label classification is therefore ill-posed for this dataset: when 94% of Place frames are also Lift, any single answer is half-wrong. This explains why Place and Lift were the two weakest classes under the original formulation.

Measured wrist kinematics match the annotation definitions, which independently validates the labelling:

Class	Wrist height (m)	Vertical velocity (m/s)	Interpretation
Place	0.754	+0.067	only class with net upward motion
Align	0.871	-0.046	highest position, not rising
Lift	0.674	+0.006	carrying

3. Experimental design

A 2x2 comparison over label encoding and augmentation, plus the original configuration as a baseline. All arms: GRU, 120-frame causal windows (4 s), 235 input features expanded to 251 (azimuth encoded as sin/cos), per-column standardisation from training subjects only, 15-fold LOSO.

Arm	Label target	Augmentation
Baseline	hard argmax (task_id)	none
Plateau / no aug	plateau vector, multi-label BCE	none
Plateau + mirror	plateau vector, multi-label BCE	mirror
Peak / no aug	peak vector, multi-label BCE	none
Peak + mirror	peak vector, multi-label BCE	mirror

Plateau is flat 1.0 across an annotated span and teaches task *presence*. **Peak** rises to 1.0 only at the span midpoint and teaches *centrality* — how far into an action the subject is.

Note on the mirror corpus: the three copies per take (_aug-01/02/03) are byte-identical — mirroring is deterministic and the varying seeds only drive rotation and noise, both unset. Using all three would weight mirrored samples 3x against originals. One copy per physical recording is retained (279 files → 93).

4. Results

4.1 Overall, 15-fold LOSO (mean ± sd)

Arm	macro-F1	bal-acc	prog MAE	lane AP	mistake F1
Baseline hard argmax	0.386 ± 0.074	0.433	25.6	—	0.147
Plateau / no aug	0.378 ± 0.066	0.412	24.5	0.527	0.163
Plateau + mirror	0.389 ± 0.075	0.420	24.1	0.536	0.170
Peak / no aug	0.383 ± 0.069	0.430	24.7	0.525	0.145
Peak + mirror	0.392 ± 0.066	0.443	24.0	0.535	0.190

Chance level for 7 classes is 0.143. Progress MAE is on a 0-100 scale. All five arms fall within 0.378-0.392, well inside the ±0.07 fold-to-fold spread — **no arm is distinguishable from another on the overall score.**

4.2 Per-class recall — where the arms actually differ

Arm	Pull	Lift	Align	Screw	Conn	Clamp	Place
Baseline hard	0.44	0.28	0.58	0.69	0.37	0.41	0.26
Plateau / no aug	0.34	0.41	0.51	0.73	0.39	0.39	0.11
Plateau + mirror	0.37	0.44	0.51	0.74	0.37	0.41	0.12
Peak / no aug	0.36	0.21	0.58	0.74	0.35	0.43	0.34
Peak + mirror	0.45	0.19	0.59	0.73	0.35	0.41	0.38

Peak targets find Place roughly three times better than plateau (0.38 vs 0.11); plateau finds Lift roughly twice as well (0.44 vs 0.19). This follows from the curve definitions: a short, nested class is buried beneath the flat plateau of its container, whereas the peak curve gives short spans proportional weight. A single headline metric conceals this entirely.

4.3 Paired significance tests

Folds are paired — every arm holds out the same 15 subjects — so per-fold differences are tested directly rather than by comparing means with overlapping error bars. Pooling both target encodings gives 30 paired folds:

Comparison	Mean delta	Wins	Wilcoxon p
Mirror augmentation (pooled)	+0.0097	22 / 30	0.058
Peak vs plateau (pooled)	+0.0039	18 / 30	0.465
Baseline → plateau / no aug	-0.0080	5 / 15	0.454

Mirror augmentation is marginally significant and small. Label encoding makes no difference to the overall score but redistributes which classes are learned.

5. Data scaling: subjects, not frames

Learning curve over the number of training subjects, fixed test set:

2 subjects	6,530 windows	macro-F1	0.252
4 subjects	11,829 windows	macro-F1	0.296
6 subjects	19,111 windows	macro-F1	0.298
8 subjects	30,696 windows	macro-F1	0.302
10 subjects	36,886 windows	macro-F1	0.343

slope over the last three points: +0.011 macro-F1 per added subject

The curve has not saturated. A controlled test of whether synthetic data substitutes for real people:

Training set	Windows	macro-F1
5 real subjects	19,160	0.288 ± 0.020
5 real + their mirror copies	38,319	0.310 ± 0.031
6 real subjects	22,672	0.316 ± 0.020

One additional real subject outperforms doubling the data with mirrored copies of five existing ones.

Augmentation of existing subjects adds no inter-person variation, which is precisely what generalisation to an unseen person requires. For any further data collection, participant count should be prioritised over footage length.

6. Event-level trigger evaluation

Frame-level macro-F1 answers whether a given frame is Align. The runtime never asks that. `task_manager.py` holds an `R_WAITING_FREE_DRIVE` state and asks permission once per assembly cycle, when the human finishes raising the panel. The corresponding transition is highly structured:

91% of Align spans begin WHILE Lift is still active
they begin at the 39-54% point of the enclosing Lift span
89% are preceded by the state (Lift + Place)

A caution for anyone extending this: the assumption that Align starts when Lift ends is wrong. After a Lift span ends, 82% of the time nothing follows, because Align lives inside Lift rather than after it.

Evaluated as event detection and pooled over all 15 folds (292 true onsets):

Threshold	Detected	Median latency	Fired early	False / min
0.3	73%	-0.3 s	56%	1.47
0.5	75%	-0.3 s	50%	1.09
0.7	75%	0.0 s	40%	0.90

75% detection at a median latency of -0.3 s, with half of all firings ahead of the annotated onset. Compare this with the frame-level Align recall of 0.51 ± 0.26 . The event framing is substantially easier because the model accumulates roughly 300 frames of evidence per event and only has to be correct once.

7. Open problem: false alarm rate

292 true events -> 640 false alarms = 2.19 per true event
one spurious prompt every 0.9 min
only 16% land within 8 s of a real onset

context during false alarms:

Screw 38% Lift 28% Align 21% Connect 12%

At roughly one unwanted prompt per minute the system would not be usable: operators habituate to declining and would reject the genuine offer as well.

7.1 Context gating was tested and does not solve it

Since Screw never co-occurs with Align, gating the Align score on Lift should remove most false alarms. Using ground-truth Lift, it does:

Trigger variant	Detected	False / event
Raw Align score	75%	2.19
Gated on ground-truth Lift	71%	0.62
Gated on predicted Lift ≥ 0.5	74%	2.03
Align x Lift (predicted)	74%	1.86
max(Align, Place) x Lift	55%	3.18

The ground-truth gate removes 72% of false alarms for three points of detection, but the deployable version, using the model own Lift prediction, removes only 15%. **The gate signal itself is unreliable:**

predicted Lift probability at TRUE detections : 0.944
predicted Lift probability at FALSE alarms : 0.727 (not separable)

at false alarms, ground-truth Lift was actually active : 28%
so 72% of false alarms genuinely fall outside Lift,
yet the model predicts Lift ≥ 0.5 there in 73% of those cases

The model over-predicts Lift. Gating logic is not the bottleneck; the gate signal is.

7.2 Root cause: the training objective

Per-lane pos_weight is set to neg/pos — Lift 3.97, Align 7.36, Pull 20.05. Any pos_weight > 1 explicitly biases binary cross-entropy toward predicting positive. **Over-prediction is the designed behaviour of the current loss.** That was the correct choice while the objective was per-class recall; it is the wrong choice now that the objective is a usable trigger.

Why this went unnoticed: every metric used so far — macro-F1, per-class recall, lane AP — is recall-weighted. None penalises a false positive, so the precision failure was invisible by construction.

8. Recommended retrain

Retrain with precision represented in the objective, then apply context gating to the improved signal. Features, windowing, normalisation, splitting and the LOSO protocol stay unchanged so results remain comparable with the five completed arms.

- **Cap pos_weight at 2.0** — Lift 3.97 to 2.0, Align 7.36 to 2.0, Pull 20.05 to 2.0. Retains partial class balancing without licensing wholesale over-prediction.

- **Add an optional focal-loss term** on the task head, which down-weights easy negatives — precisely the confidently-wrong-Lift case driving the false alarms.
- **Report per-lane precision** at the operating threshold. This is the metric that was missing; without it the next run is blind in the same way.
- **Then gate:** $P(\text{Align}) \times P(\text{Lift})$, fired on a sustained crossing, at most once per cycle — matching the triggered_keys deduplication already present in `trigger_manager.py`, which the offline evaluation does not yet model and which will further reduce the practical rate.

Three arms, 15-fold LOSO each, on peak + mirror (the strongest current configuration): capped, capped + focal, and uncapped as a control.

8.1 Expected outcome

Frame-level macro-F1 will probably fall slightly. Capping pos_weight trades recall for precision, and macro-F1 is recall-weighted. Recall for the rare classes (Pull, Place) will drop most.

The target is the trigger metric: false alarms moving from 2.19 toward the 0.62 ceiling with detection held near 70-75%. **This may not be achieved.** The ceiling assumes a perfect Lift signal; a capped model will be better but not perfect, so a realistic landing zone is 1.0-1.4 false alarms per event — a genuine improvement that may still be too many. In that case the honest conclusion is that this trigger requires a confirmation step or a second sensing modality, not a better classifier.

9. Conclusions

- **The evaluation protocol was the largest single source of error.** A leaking split reported 0.806 where the honest figure is 0.298. Subject-disjoint LOSO is the minimum defensible protocol for this dataset.
- **Architecture is not the bottleneck.** Measured split-to-split spread (0.085) is 3.3x the entire gap between GRU, LSTM, 2-layer LSTM and TCN (0.026). Comparisons on a single split cannot distinguish them.
- **Label encoding redistributes rather than improves.** Peak and plateau targets score identically overall but learn different classes — peak recovers Place (0.38 vs 0.11), plateau recovers Lift (0.44 vs 0.19).
- **Data scaling is limited by subjects, not frames.** Each additional real subject is worth about +0.011 macro-F1 and the curve has not saturated, while synthetic copies of existing subjects underperform a single new participant.
- **The system-level result is stronger than the classification result.** 75% event detection at −0.3 s median latency is a usable proactive signal; frame-level macro-F1 of 0.39 understates it because the robot never needs a per-frame answer.
- **Precision is the remaining obstacle, and it is addressable.** The false-alarm rate stems from a known property of the loss function, not from an inherent limit of the data.

Appendix: reproduction

All code is under `bench/`. The pipeline is deterministic given a fixed seed.

File	Role
<code>bench/data.py</code>	subject-disjoint split, background mask, augmentation dedup
<code>bench/engine.py</code>	shared training and evaluation loop

bench/loso.py	15-fold leave-one-subject-out cross-validation
bench/trigger_eval.py	event-level trigger metric
bench/compare.py	arm comparison with paired Wilcoxon tests
bench/verify.py	pre-flight assertions (run before any training)

verify.py asserts: background-mask exactness (100.00% over 286,123 frames), the progress-vector gather identity (max difference 0.000), absence of augmentation leakage across splits, and correct deduplication of mirror copies (279 to 93).